Работа с табличными данными

Как превратить сырую таблицу в признаки для модели:

EDA, пропуски, выбросы, категории, масштаб, даты

Преподаватель: **Вересников Артем**, MLE в WB

Что такое табличные данные

id	площадь	комнаты	район	этаж	год дома	дата продажи	цена, млн
1	54	2	Центр	5	2015	12.03.2024	12.5
2	38	1	Север	NaN	1978	03.04.2024	6.1
3	1	3	Юг	2	2003	15.04.2024	9.8
4	72	3	Центр	12	NaN	20.05.2024	18.0
5	45	2	север	3	1990	20.05.2024	7.4

Строка = объект

Одна квартира. Объекты независимы друг от друга.

Столбец = признак

Свойство объекта: число, категория, дата, текст.

Таргет

То, что предсказываем: здесь цена. В test его нет.

Типы признаков

тип	пример	что делать
Числовой	площадь, этаж, цена	масштабировать, проверить хвосты и выбросы
Категориальный	район, марка машины	закодировать в числа
Порядковый	образование, размер S/M/L	номера по порядку или one-hot
Бинарный	есть балкон: да / нет	0 и 1
Дата / время	дата продажи	разобрать на год, месяц, «сколько дней прошло»
Текст	описание объявления	длина, ключевые слова — отдельная тема
ID	id, номер заказа	выбросить: смысла нет, может быть утечкой

Первый вопрос к каждому столбцу: **какого он типа на самом деле?** Число в столбце ещё не значит, что признак числовой.

Первый взгляд: pandas

```
import pandas as pd

df = pd.read_csv('flats.csv')

df.shape        # (строк, столбцов)
df.head()       # первые 5 строк
df.info()       # типы и непустые
df.describe()   # статистики чисел
df.isna().sum() # пропуски
df['район'].value_counts()
```

info()

Показывает тип каждого столбца. Год записан как object? Значит, внутри мусор или пропуски-строки.

describe()

min, max, среднее, медиана. Площадь min = 1 — повод насторожиться.

value_counts()

«Север» и «север» — две категории. Нужна чистка.

Чек-лист первого взгляда

проблема	как увидеть	пример
Пропуски	df.isna().sum()	этаж не указан
Дубликаты	df.duplicated().sum()	одно объявление дважды
Неверный тип	df.dtypes	год как строка «2003г»
Ошибки ввода	describe(), логика	площадь 1 м², этаж 30 в 5-этажке
Опечатки в категориях	value_counts()	«Север», «север», «Север »
Константы	df.nunique()	столбец, где везде «Москва»
Выбросы и хвосты	гистограмма, перцентили	цена 900 млн

Сначала **находим** все проблемы, потом решаем, что с каждой делать.

Срезы и группировки

```
# фильтр
df[df['комнаты'] == 2]

# средняя цена по району
df.groupby('район')['цена'].agg(['mean','count'])

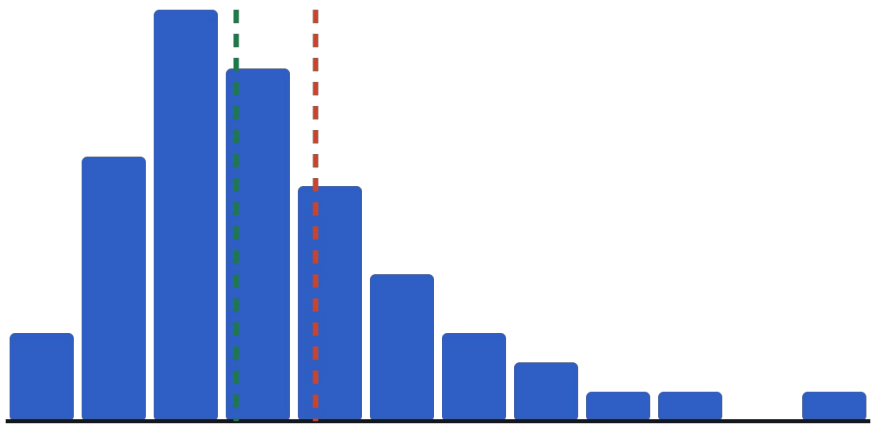
# соединить с другой таблицей
df.merge(districts, on='район', how='left')
```

Результат groupby:

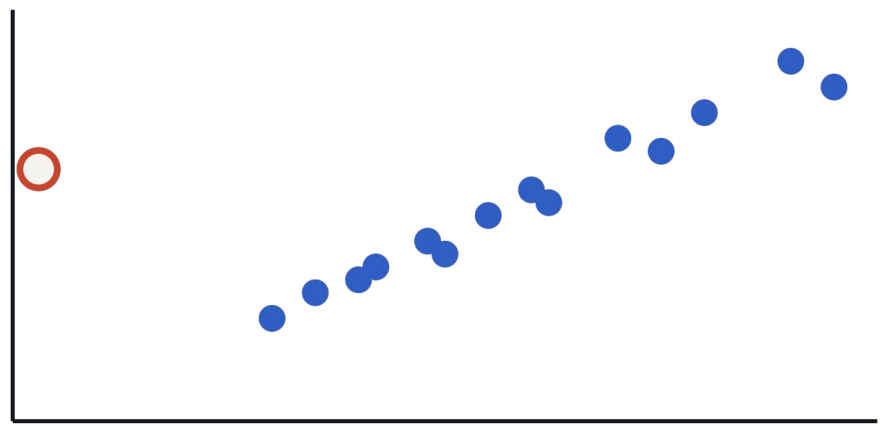
район	mean	count
Центр	15.3	2
Север	6.8	2
Юг	9.8	1

Группировки — главный инструмент EDA и основа признаков-агрегатов: «средняя цена в районе».

Смотрим глазами

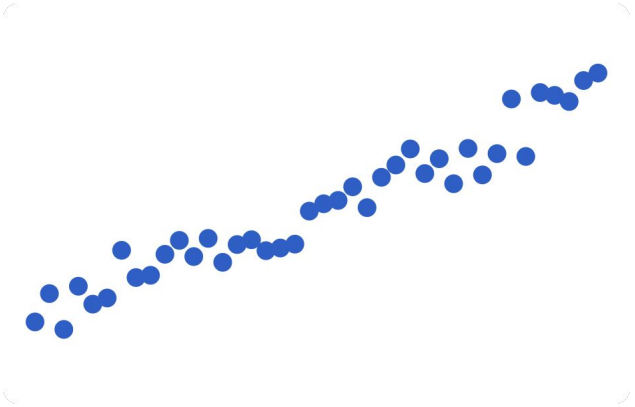


Гистограмма цены. Длинный хвост вправо: среднее правее медианы.



Площадь vs цена. Почти линейно — и одна точка-ошибка: площадь 1 м².

Связь с таргетом: числовые признаки

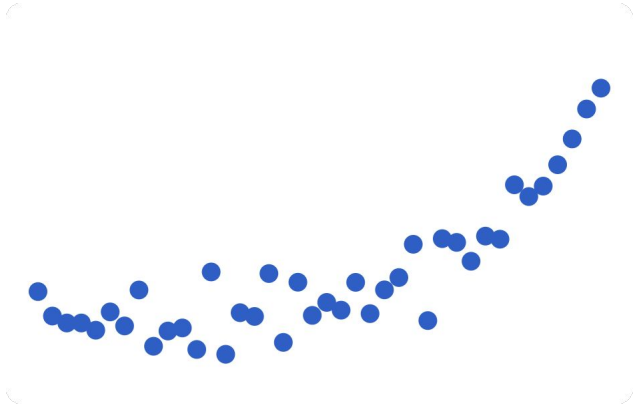


Прямая

`Пирсон = 0.97`

`Спирмен = 0.97`

Оба видят связь

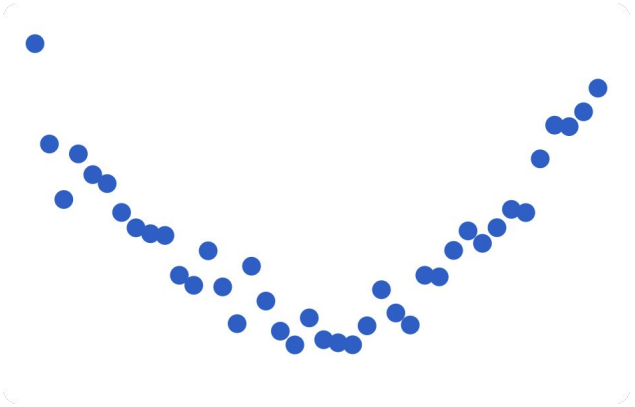


Растёт, но не прямой

`Пирсон = 0.85`

`Спирмен = 0.98`

Спирмен видит лучше



Парабола

`Пирсон = ≈ 0`

`Спирмен = ≈ 0`

Оба слепы — смотри график

Пирсон — насколько точки ложатся на **прямую**.

Спирмен — насколько связь **монотонная**: растёт x → растёт y.

Классификация? Рисуем распределение признака отдельно для класса 0 и класса 1. Не пересекаются — признак сильный.

Связь с таргетом: категории

Регрессия: средний таргет в группе

`df.groupby('район')['цена'].agg(['mean','count'])`

район	средняя цена	объектов
Центр	15.3 млн	420
Север	6.8 млн	380
Юг	9.8 млн	3

Средние сильно различаются → признак полезен. Но Юг — всего 3 объекта: такому среднему верить нельзя.

Классификация: доля класса 1 в группе

`df.groupby('тариф')['ушёл'].agg(['mean','count'])`

тариф	доля ушедших	клиентов
помесячный	43%	2100
годовой	11%	1500
двухлетний	3%	900

Общая доля 24%. Помесячный — почти вдвое выше среднего: сильный признак.

Смотрим на **разброс средних между группами**: все примерно одинаковые — признак бесполезен. Всегда рядом с средним — **размер группы**.

Золотое правило предобработки

Все числа, которые мы считаем по данным, — только по train

- медианы и средние для пропусков
- μ и σ для StandardScaler, min и max
- перцентили для клиппинга
- словари категорий для one-hot
- частоты и средний таргет для кодировок
- средние по группам для признаков

Посчитали на train → применили **те же самые числа** к val и test. Не пересчитываем!

Пропуски: сначала понять почему

Не измерили

Датчик сломался, человек не заполнил. Можно заполнить типичным значением.

Не бывает

Нет второго телефона, нет ипотеки. Пропуск значит «0» или отдельная категория.

Скрыли

Не указал доход — и это само по себе сигнал. Нужен индикатор пропуска.

стратегия	когда
Удалить строки	пропусков мало и они случайны
Удалить столбец	пропусков больше 70–80%
Медиана / мода	числа / категории — простой базовый вариант
Константа или «missing»	пропуск значит «не бывает»
+ индикатор «был пропуск»	сам факт пропуска может предсказывать

Медиана или среднее?

Площади в train:

30, 40, 45, 50, 1000

1000 — ошибка ввода (лишние нули)

$$\text{среднее} = \frac{30 + 40 + 45 + 50 + 1000}{5} = 233$$

медиана = 45

Заполнить пропуск площадью 233 м² — абсурд. Медиана почти не замечает один выброс.

Дубликаты и ошибки ввода

```
# дубликаты
df.duplicated().sum()
df = df.drop_duplicates()

# опечатки в категориях
df['район'] = (df['район']
.str.strip().str.lower())

# логические проверки
bad = df['площадь'] < 10
df.loc[bad, 'площадь'] = None
```

Ошибка → пропуск

Площадь 1 м² не удаляем вместе со строкой: превращаем в пропуск, дальше его заполнит общий механизм.

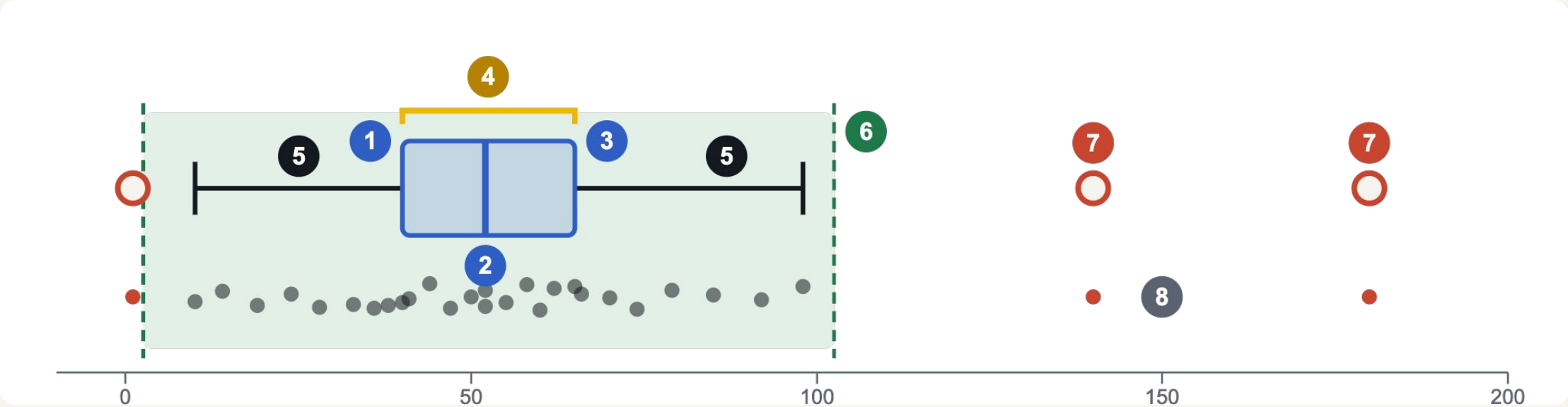
Логика предметной области

Этаж больше этажности дома, дата продажи раньше постройки, возраст 150 лет.

Дубликаты и валидация

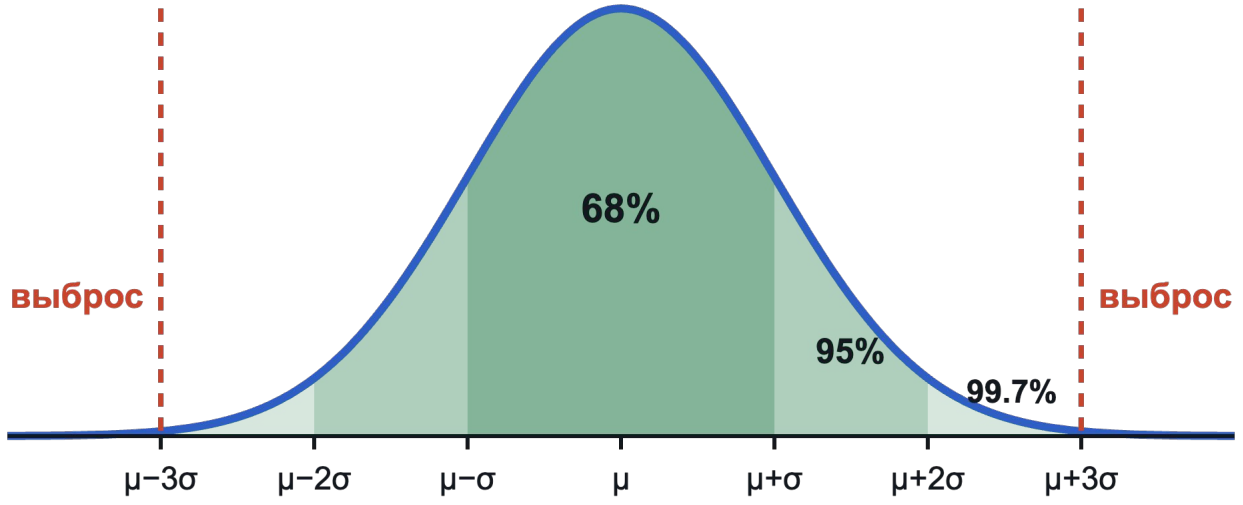
Копия объекта в train и test завышает качество.

Выбросы: как читать «ящик с усами»



1. **Q1 = 40**: четверть квартир меньше
2. **Медиана = 52**: половина меньше, половина больше
3. **Q3 = 65**: три четверти квартир меньше
4. **IQR = Q3 − Q1 = 25**: ширина ящика, тут средние 50%
5. **Усы**: от ящика до самой дальней точки внутри границ — 10 и 98
6. **Границы**: Q1 − 1.5·IQR = 2.5 и Q3 + 1.5·IQR = 102.5
7. **Выбросы**: всё за границами — 1, 140, 180 м²
8. **Сами данные**: каждая точка — одна квартира

Правило трёх сигм



$$|x - \mu| > 3\sigma \rightarrow \text{выброс}$$

то есть z-оценка (x − μ) / σ по модулю больше 3

Пример

Площади: μ = 55, σ = 15. Границы: 55 ± 45 → от 10 до 100 м².

Когда работает

Распределение похоже на колокол: рост, вес, ошибки измерений.

Когда врёт

Хвост (цены, доходы): σ раздута самими выбросами. Тогда — IQR или сначала log.

Кодирование категорий

способ	как	когда
One-hot	столбец 0/1 на каждую категорию	мало категорий; линейные модели
Порядковый	S → 0, M → 1, L → 2	есть естественный порядок
Частотный	категория → её доля в train	много категорий
Target encoding	категория → средний таргет в ней	много категорий; только внутри фолда!

Редкие категории

Объединяем в «other»: вес по 2 объектам переобучается.

Новые в test

Категории, которой не было в train: handle_unknown='ignore' — все нули.

Утечка

Target encoding по всему train «подсказывает» модели ответ.

Масштабирование

StandardScaler

$$x' = \frac{x - \mu}{\sigma}$$

среднее 0, разброс 1

MinMaxScaler

$$x' = \frac{x - \min}{\max - \min}$$

всё в [0, 1]

RobustScaler

$$x' = \frac{x - \text{медиана}}{\text{IQR}}$$

устойчив к выбросам

Зачем

Площадь ~ 50, этаж ~ 5, цена ~ 10^7. Градиентный спуск и регуляризация чувствительны к масштабу.

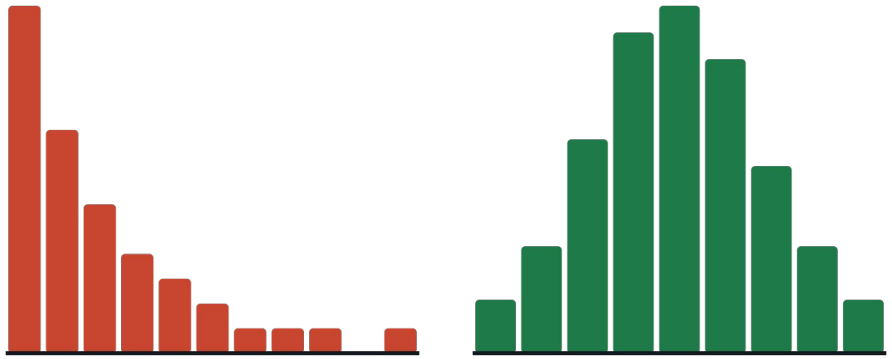
Кому не нужно

Деревьям и бустингу: им важен только порядок значений.

Главное правило

fit — только на train. Тот же μ и σ применяем к test.

Преобразования признаков



до: длинный хвост → **после** log(1 + x): почти симметрично

log(1 + x)

Для хвостатых признаков: цены, доходы, число просмотров. +1 — чтобы не брать log 0.

Биннинг

Возраст → интервалы 0–18, 18–30, … + one-hot. Даёт линейной модели «ступеньки».

Взаимодействия

площадь × этаж, x^2: линейная модель сама их не придумает.

Даты и время

```
d = pd.to_datetime(df['дата продажи'],
                   dayfirst=True)

df['год']   = d.dt.year
df['месяц'] = d.dt.month
df['день_недели'] = d.dt.dayofweek
df['возраст_дома'] = d.dt.year - df['год дома']
```

Дата — не число

Модель не поймёт «12.03.2024». Разбираем на части.

Разности

«Сколько дней прошло», возраст дома — часто полезнее самой даты.

Цикличность

Декабрь (12) и январь (1) рядом. Кодируем через sin и cos месяца.

Новые признаки из головы

Отношения

цена за м² у соседей, площадь на комнату, платёж / доход.

Агрегаты по группам

средняя площадь в районе, число объявлений в доме.

Флаги

первый этаж, последний этаж, новостройка (возраст < 3 лет).

```
df['м2_на_комнату'] = df['площадь'] / df['комнаты']
df['первый_этаж']   = (df['этаж'] == 1).astype(int)
df['ср_площадь_района'] = df.groupby('район')['площадь'].transform('mean')
```

Каждый новый признак проверяем на валидации: **оставляем, только если стало лучше**.

Train, val и test



Train — учим

- веса модели
- медианы, μ и σ, кодировки

Val — настраиваем

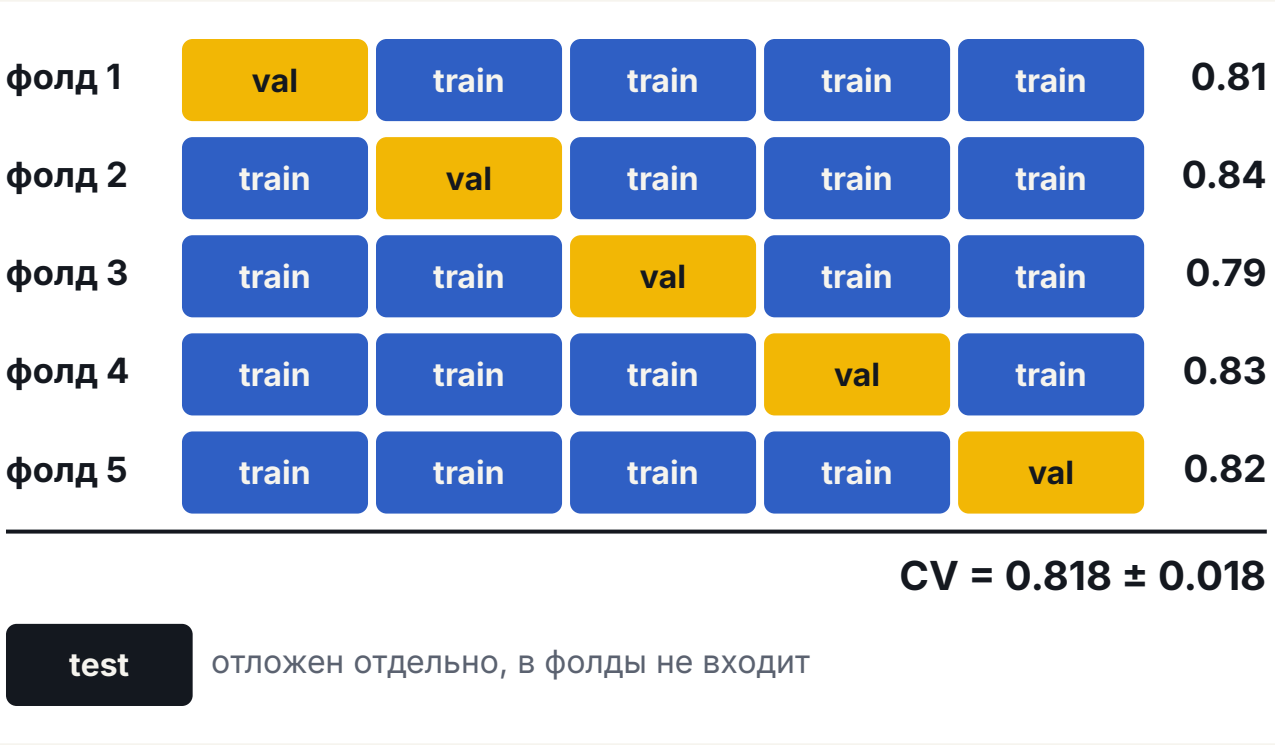
- гиперпараметры: λ, C, глубина
- порог вероятности
- какие признаки оставить
- какую модель выбрать

Test — проверяем

- один раз, в самом конце
- ничего по нему не подбираем
- честная оценка «как в проде»

Подобрали порог или λ по test — test превратился в val, честной оценки больше нет.

Кросс-валидация



Как

Делим train на K частей. K раз учим на K − 1, проверяем на оставшейся. Каждый объект побывал в val ровно раз.

Зачем

Один val шумный: повезло или нет с объектами. Среднее по 5 фолдам надёжнее, ± показывает разброс.

Правило

Предобработка учится заново в каждом фолде — на его train-части. Pipeline делает это сам.

Как делить на фолды

StratifiedKFold



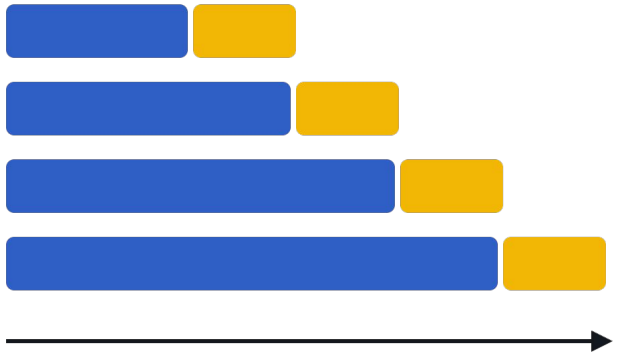
класс 0 · класс 1 — в каждом фолде 80 / 20

GroupKFold



цвет = группа; группа целиком в одном фолде

TimeSeriesSplit



train · val → время

Стратификация

Та же доля классов в каждом фолде. Важно при дисбалансе.

Группы

Объекты одного клиента, дома, пациента — в одном фолде, если в test новые группы.

Время

Если test — будущее, валидация тоже должна быть позже обучения.

Правило: валидация повторяет то, как разделены train и test. Обычный KFold при новых клиентах или будущем периоде даёт завышенный CV.

Утечки при предобработке

Неправильно

1. Считаем медиану и μ, σ по **всем** данным
2. Делим на train, val, test
3. Учим и проверяем

Правильно

1. Делим на train, val, test
2. Считаем медиану, μ, σ, кодировки **только по train**
3. Применяем те же числа к val и test

Почему это важно

Статистики val и test «подтекли» в обучение — оценка качества завышена, на новых данных будет хуже.

Правило

fit — на train, transform — на train, val и test. Всё, что учится на данных, — внутри train.

Pipeline: всё одним вызовом

```
num = make_pipeline(
    SimpleImputer(strategy='median',
                  add_indicator=True),
    StandardScaler())
cat = make_pipeline(
    SimpleImputer(strategy='most_frequent'),
    OneHotEncoder(handle_unknown='ignore'))

prep = ColumnTransformer([
    ('num', num, num_cols),
    ('cat', cat, cat_cols)])
model = make_pipeline(prep, LogisticRegression())
model.fit(X_train, y_train)
```

Зачем

Один объект делает всю предобработку и модель. fit на train сам выучит медианы, μ, σ и категории.

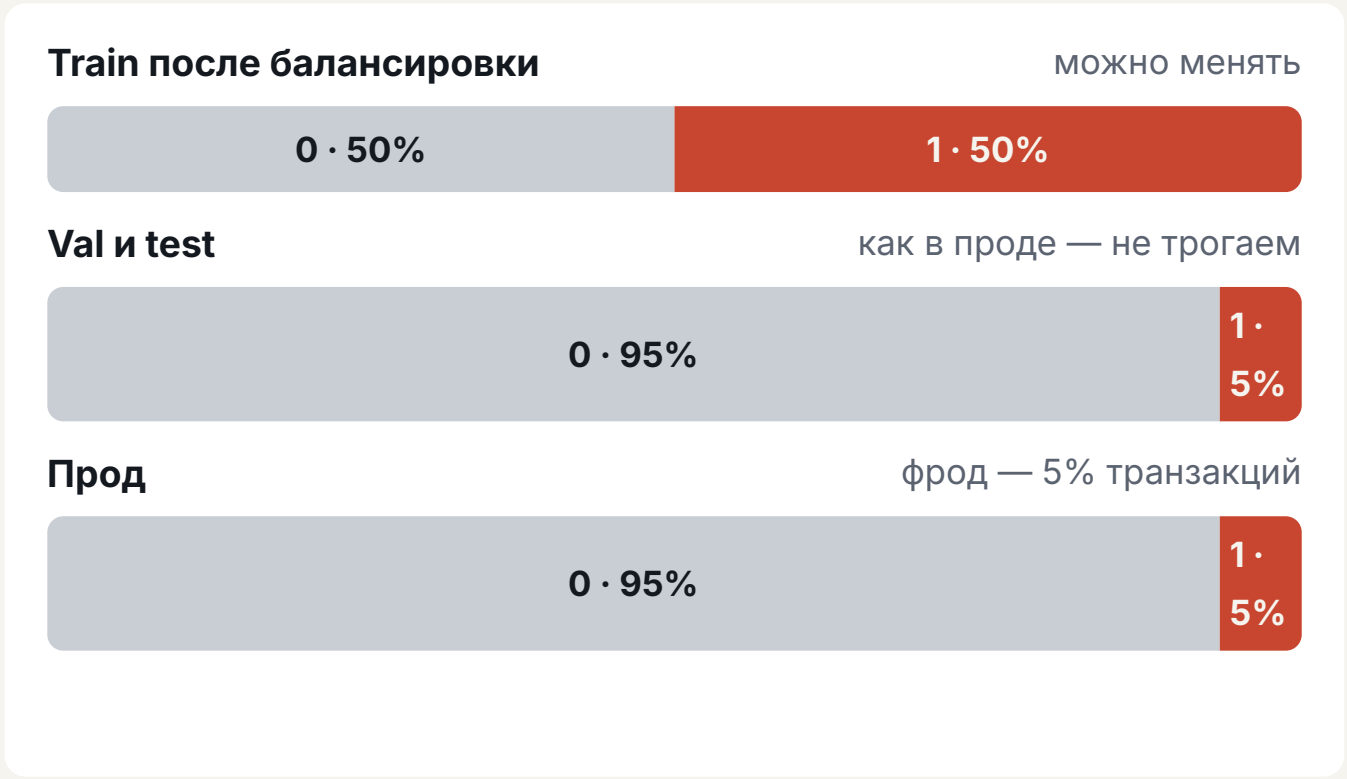
Нет утечек

В кросс-валидации предобработка автоматически учится заново внутри каждого фолда.

Одинаково для test

model.predict(X_test) применит те же шаги в том же порядке.

Дисбаланс классов



Undersampling

Выкидываем часть объектов частого класса.

Oversampling

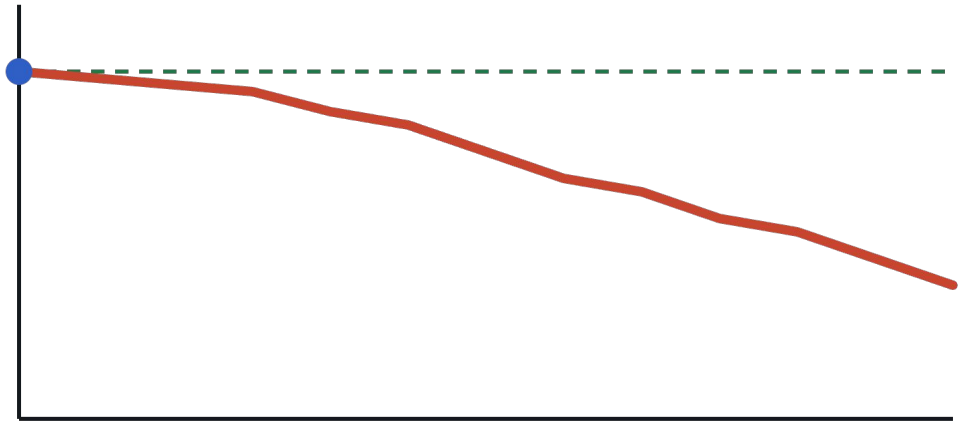
Копируем редкий класс. SMOTE — создаёт новых «соседей».

Веса классов

class_weight — редкий класс важнее в лоссе.

Балансируем **только train**. Val и test — с тем распределением, что будет в проде, иначе метрики и порог соврут.

Модели портятся: дрейф данных



пунктир — качество на test при запуске, линия — в проде, 12 месяцев

Дрейф данных

Меняются признаки: цены выросли из-за инфляции, пришли новые районы.

Дрейф концепции

Меняется связь признак → ответ: мошенники придумали новую схему.

Что делать

Следить за распределениями и метрикой, регулярно переобучать на свежих данных.

Порядок предобработки

1. Посмотреть: info, describe, графики
2. Почистить: дубли, опечатки, ошибки → NaN
3. Разделить: train / val / test
4. Пропуски: медиана, «missing», индикатор
5. Выбросы и хвосты: клиппинг, log
6. Категории: one-hot, частотное
7. Даты и новые признаки
8. Масштаб: StandardScaler
9. Pipeline → настроить на val → один раз на test

Начиная с шага 4 всё учится **только на train** — и живёт внутри Pipeline.